

1. Visão Geral

O **HarleyDavidsonStore** é um sistema integrado de gestão voltado para concessionárias e oficinas mecânicas especializadas em motocicletas. O projeto centraliza o controle de estoque de peças e veículos, cadastros de clientes, histórico de faturamento/transações, além do gerenciamento completo do fluxo de ordens de serviço (OS) da oficina mecânica.

2. O Problema

Concessionárias e oficinas de grande porte frequentemente enfrentam descontrole no fluxo de peças, falta de rastreabilidade de serviços prestados por moto/cliente e ruídos no acompanhamento de transações financeiras (vendas diretas vs. serviços). A falta de um domínio unificado resulta em divergência de estoque e insatisfação do cliente.

3. Objetivos Estratégicos

- **Unificação:** Unificar o controle de vendas no balcão, vendas de motocicletas e prestação de serviços mecânicos em uma única plataforma.
- **Rastreabilidade:** Garantir a rastreabilidade total do histórico das motos dos clientes em relação aos serviços realizados e peças aplicadas.
- **Gestão de Estoque:** Manter controle rígido de movimentação de estoque (compras de fornecedores versus saídas por vendas ou Ordens de Serviço).
- **Visão Financeira:** Oferecer uma visão consolidada das operações financeiras e entradas de mercadorias para melhor gestão do negócio.

4. Público-Alvo e Usuários

Vendedores

Responsáveis por transações de vendas no balcão, negociações de motocicletas e atendimento direto.

Mecânicos

Responsáveis pela execução técnica dos serviços e alocação de peças nas Ordens de Serviço (OS).

Gerentes

Responsáveis pelo acompanhamento global, aprovações, gestão de fornecedores e controle financeiro da unidade.

Clientes

Proprietários das motocicletas cadastradas no sistema, consumidores de produtos e serviços.

5. Escopo e Principais Funcionalidades

O sistema abrangerá os seguintes módulos de gestão e funcionalidades:

- **Cadastros Base:** Consulta e manutenção de fornecedores, produtos, clientes, funcionários e motocicletas.
- **Gestão de Suprimentos:** Registro de entrada de mercadorias (composição do estoque) atualizando o valor unitário e total.
- **Catálogo e Precificação:** Gestão do catálogo de produtos e peças, monitorando rigorosamente o saldo de estoque e preço de venda.
- **Gestão de Serviços (OS):** Abertura, gerenciamento do ciclo de vida (ABERTA, EM_ANDAMENTO, CONCLUIDA, CANCELADA), fechamento e associação de itens/quantidades.
- **Transações e Financeiro:** Lançamento de transações comerciais categorizadas (MOTO, PEÇAS, BALCÃO, COMPRA, ORDEM_SERVIÇO) e visão consolidada.

6. Requisitos e Restrições Importantes

- **Unicidade:** Obrigatória para identificadores fiscais e físicos (CNPJ de fornecedores, CPF/CNPJ de clientes, Placa e Chassi das motos).
- **Estoque e Valores:** O estoque de produtos não pode ser negativo ($\text{estoque_qtd} \geq 0$). Preços e quantidades devem ser estritamente positivos (> 0).
- **Restrição de Domínio:** Tipos de funcionários são restritos a VENDEDOR, MECANICO ou GERENTE.

7. Modelo de Domínio (Cadastros Base)

Este modelo especifica as entidades, atributos conceituais e regras de negócio que regem a base de dados da concessionária e oficina.

Entidade	Atributos Principais	Regras & Relacionamentos
Fornecedores Parceiros de reposição	Identificador (id_fornecedor), Nome/Razão Social, CNPJ, Contato.	O CNPJ deve ser único no sistema.
Produtos Itens físicos/Peças	Identificador (id_produto), Nome, Descrição, Categoria, Quantidade em Estoque, Preço de Venda.	O saldo em estoque e o preço de venda não podem ser negativos .
Funcionarios Colaboradores da loja	Identificador (id_funcionario), Nome, Cargo, Tipo, Contato.	O tipo deve ser: VENDEDOR , MECANICO ou GERENTE .
Clientes Proprietários/Compradores	Identificador (id_cliente), Nome, CPF/CNPJ, Telefone, Email, Endereço.	CPF ou CNPJ deve ser único no sistema.
Motos_Clientes Veículos vinculados	Identificador (id_moto_cliente), Cliente Vinculado, Modelo, Placa, Chassi.	Placa e Chassi devem ser únicos . Pertence a um Cliente.

8. Modelo de Domínio (Operações)

Entidade	Atributos Principais	Regras & Relacionamentos
Entrada_Mercadoria & Itens_Compra Abastecimento	<i>Entrada:</i> id_entrada, id_fornecedor, Data, Valor Total. <i>Itens:</i> id_item_compra, Produto, Quantidade, Valor Un.	Uma entrada possui itens e pertence a um Fornecedor.
Ordens_Servico & Itens_OS Atendimento Técnico	<i>OS:</i> id_os, id_moto_cliente, id_funcionario, Data, Status. <i>Itens:</i> id_item_os, Produto, Quantidade, Valor Total.	Status: ABERTA , EM_ANDAMENTO , CONCLUIDA , CANCELADA .
Transacoes Fluxo Financeiro	id_transacao, Tipo, Funcionário, Cliente, Moto, Data, Valor Total.	Tipos: MOTO , PECAS , BALCAO , COMPRA , ORDEM_SERVICO .

Visão Consolidada (vw_resumo_operacoes)

O domínio conta com um consolidado conceitual de operações que unifica as Transações Comerciais e as Entradas de Mercadorias para relatórios e consultas gerais de fluxo operacional.

9. Arquitetura Tecnológica e Princípios

- **Banco de Dados:** Microsoft SQL Server 2019 (esquema físico HarleyDavidsonStore).
- **Backend / API:** A ser definido no ciclo incremental (ex: Node.js / C# .NET / Python).
- **Frontend:** A ser definido no ciclo incremental (ex: React / Angular).

Princípios de Desenvolvimento:

- As regras de integridade do banco (Foreign Keys, Uniques e Checks) devem ser sempre espelhadas nas validações das camadas de aplicação.
- A integridade do estoque deve ser priorizada rigorosamente durante a baixa via OS ou Vendas no Balcão.
- Mudanças devem ser executadas utilizando o OpenSpec de forma incremental.

10. Organização e Estrutura do Projeto

A estrutura de diretórios foi desenhada para permitir que novas funcionalidades sejam acompanhadas do início ao fim de forma lógica e centralizada.

```
LOJAHARLEY/  
├── LOJAHARLEY/      # Código da aplicação Reflex (ponto de entrada)  
├── assets/          # Imagens e arquivos estáticos (recursos UI)  
├── docs/            # Documentação funcional, técnica e visão geral  
├── integrations/    # Integrações e exports de sistemas externos  
│   └── xano/        # Exports XanoScript do backend Xano  
├── openspec/        # Especificações, designs e mudanças incrementais  
├── .vscode/         # Configurações locais e de ambiente do VS Code  
├── requirements.txt  # Dependências Python para execução do projeto  
└── rxconfig.py      # Arquivo principal de configuração do Reflex
```

Nota: Pastas geradas automaticamente (como `.venv/`, `.states/`, `.web/`, `__pycache__/` e `reflex.lock/`) são gerenciadas por ferramentas e não devem receber código manual.

11. Fluxo de Trabalho e Setup

Ciclo de Desenvolvimento

1. Entenda o problema via `docs/project-overview.md`.
2. Confira regras em `docs/domain-model.md`.
3. Especifique mudanças em `openspec/changes/`.
4. Altere código em `LOJAHARLEY/` (assets em `assets/`).
5. Valide com `python -m py_compile` e atualize docs.

Como Começar (Setup)

1. Ative o ambiente virtual:

```
.\.venv\Scripts\Activate.ps1
```

2. Instale dependências:

```
pip install -r requirements.txt
```

3. Inicie o sistema Reflex:

```
reflex run
```